DEVELOPER GUIDE

CittaEFS API Documentation

Integration reference for authentication, invoice submission, webhooks and payment workflows.

BASE URL

`https://ei-api.azurewebsites.net`

Code examples: C#

1. Authentication

All requests require an API Key passed via the Bearer token header.

```
Authorization: Bearer <YOUR_API_KEY>
```

```
using var client = new HttpClient();
client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", "YOUR_API_KEY");

var response = await client.GetAsync(
    "${ApiConfig.baseUrl}/api/einvoice/archive");
```

2. Webhooks Configuration

Webhook notifications are sent as POST requests to your configured URL for the following events:

- invoice.created
- invoice.signed
- invoice.transmitted
- invoice.payment.updated
- invoice.validation.failed

3. Receiving Webhooks

When an event occurs, CittaEFS sends a POST request to your configured webhook URL. Your application should verify the webhook signature before processing the payload.

Webhook Headers

```
X-Webhook-Signature: <HMAC-SHA256 signature>
X-Webhook-Id: <unique webhook ID>
X-Webhook-Event: <event type>
Content-Type: application/json
```

The following headers are included with webhook requests:

X-Webhook-Signature
HMAC-SHA256 signature generated using your webhook secret.

X-Webhook-Id
Unique identifier for the webhook delivery.

X-Webhook-Event
Identifies the webhook event type.

Signature Validation

The X-Webhook-Signature value is generated using HMAC-SHA256 with your webhook secret and the exact raw JSON request body.

```
HMAC-SHA256(
    webhook secret,
    exact raw request body
)

Result:
    lowercase hexadecimal string

Example:

X-Webhook-Signature:
a3f5c8e9...64 hexadecimal characters
```

The signature is lowercase hexadecimal. It is not Base64 encoded and does not contain a sha256= prefix.

```
X-Webhook-Signature: YOUR_64_CHARACTER_HEX_SIGNATURE
```

Webhook Payload

```
{
  "event": "invoice.transmitted",
  "timestamp": "2026-07-28T10:30:00.000Z",
  "data": {
    "invoiceNumber": "INV2024001",
    "irn": "IRN12345678901234567890",
    "sourceNbr": "PO-001",
    "status": "TRANSMITTED",
    "qrCode": "iVBORw0KGgoAAAANSUhEUgAA...",
    "transmittedAt": "2026-07-28T10:30:00.000Z",
    "signedAt": "2026-07-28T10:25:00.000Z",
    "paymentStatus": "PENDING",
    "paymentReference": null,
    "totalAmount": 3117500.00,
    "currency": "NGN",
    "errorMessage": null,
    "eventType": "invoice.transmitted",
    "metadata": {
      "CustomField1": "Value1",
      "CustomField2": "Value2"
```

```
    }
  },
  "webhookMetadata": {
    "companyId": "550e8400-e29b-41d4-a716-446655440000",
    "webhookId": "123e4567-e89b-12d3-a456-426614174000",
    "attempt": 1
  }
}
```

C# / ASP.NET Core

```
using System.Security.Cryptography;
using System.Text;

[ApiController]
[Route("api/webhooks")]
public class WebhookController : ControllerBase
{
    private const string WebhookSecret =
        "YOUR_WEBHOOK_SECRET";

    [HttpPost]
    public async Task<IActionResult> ReceiveWebhook()
    {
        Request.EnableBuffering();

        using var reader = new StreamReader(
            Request.Body,
            Encoding.UTF8,
            leaveOpen: true
        );
```

```
        var payload = await reader.ReadToEndAsync();

        Request.Body.Position = 0;

        var signature =
            Request.Headers["X-Webhook-Signature"]
                .FirstOrDefault();

        if (string.IsNullOrWhiteSpace(signature))
            return Unauthorized(new
            {
                error = "Missing webhook signature"
            });

        using var hmac = new HMACSHA256(
            Encoding.UTF8.GetBytes(WebhookSecret)
        );

        var hash = hmac.ComputeHash(
```

```
            Encoding.UTF8.GetBytes(payload)
        );

        var expectedSignature =
            Convert.ToHexString(hash)
                .ToLowerInvariant();

        if (!signature.Equals(
                expectedSignature,
                StringComparison.OrdinalIgnoreCase))
        {
            return Unauthorized(new
            {
                error = "Invalid webhook signature"
            });
        }

        // Signature is valid.
        // Deserialize and process the payload.
```

```
        var webhook =
            System.Text.Json.JsonSerializer
                .Deserialize<WebhookPayload>(payload);

        // Process webhook...

        return Ok(new
        {
            success = true
        });
    }
}
```

Node.js / Express

```
const express = require("express");
const crypto = require("crypto");

const app = express();

const WEBHOOK_SECRET =
  "YOUR_WEBHOOK_SECRET";

// IMPORTANT:
// Use the raw body for signature validation.

app.post(
  "/api/webhooks",
  express.raw({
    type: "application/json"
  }),
  (req, res) => {

    const signature =
      req.header("X-Webhook-Signature");
```

```


    if (!signature) {
      return res.status(401).json({
        error: "Missing webhook signature"
      });
    }

    const payload = req.body;

    const expectedSignature = crypto
      .createHmac(
        "sha256",
        WEBHOOK_SECRET
      )
      .update(payload)
      .digest("hex");

    if (
      signature.length !==
        expectedSignature.length ||
```

```
      !crypto.timingSafeEqual(
        Buffer.from(signature),
        Buffer.from(expectedSignature)
      )
    ) {
      return res.status(401).json({
        error: "Invalid webhook signature"
      });
    }

    // Signature is valid.

    const webhook =
      JSON.parse(
        payload.toString("utf8")
      );

    // Process webhook...

    return res.status(200).json({
```

```
      success: true
    });
  }
);
```

Python / Flask

```
from flask import Flask, request, jsonify
import hashlib
import hmac

app = Flask(__name__)

WEBHOOK_SECRET =
    "YOUR_WEBHOOK_SECRET"


@app.route(
    "/api/webhooks",
    methods=["POST"]
)
def receive_webhook():

    signature = request.headers.get(
        "X-Webhook-Signature"
    )
```

```
    if not signature:
        return jsonify({
            "error":
                "Missing webhook signature"
        }), 401

    # Get the exact raw request body.

    payload = request.get_data()

    expected_signature = hmac.new(
        WEBHOOK_SECRET.encode("utf-8"),
        payload,
        hashlib.sha256
    ).hexdigest()

    if not hmac.compare_digest(
        signature.lower(),
        expected_signature.lower()
    ):
```

```
        return jsonify({
            "error":
                "Invalid webhook signature"
        }), 401

    # Signature is valid.

    webhook = request.get_json()

    # Process webhook...

    return jsonify({
        "success": True
    }), 200
```

PHP

```
<?php

$webhookSecret =
    "YOUR_WEBHOOK_SECRET";

$signature =
    $_SERVER["HTTP_X_WEBHOOK_SIGNATURE"]
    ?? "";

if (empty($signature)) {
    http_response_code(401);

    echo json_encode([
        "error" =>
            "Missing webhook signature"
    ]);

    exit;
}
```

```
// Get the exact raw request body.

$payload =
    file_get_contents("php://input");

$expectedSignature = hash_hmac(
    "sha256",
    $payload,
    $webhookSecret
);

if (!hash_equals(
    strtolower($expectedSignature),
    strtolower($signature)
)) {
    http_response_code(401);

    echo json_encode([
        "error" =>
            "Invalid webhook signature"
    ]);

    exit;
}

// Signature is valid.

$webhook =
    json_decode($payload, true);

// Process webhook...

http_response_code(200);

echo json_encode([
    "success" => true
]);
```

Java

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.nio.charset.StandardCharsets;

public class WebhookValidator {

    public static boolean validateSignature(
            String payload,
            String signature,
            String secret)
            throws Exception {

        Mac mac =
            Mac.getInstance("HmacSHA256");

        SecretKeySpec secretKey =
            new SecretKeySpec(
                secret.getBytes(
                    StandardCharsets.UTF_8
                ),
```

```
                "HmacSHA256"
            );

        mac.init(secretKey);

        byte[] hash =
            mac.doFinal(
                payload.getBytes(
                    StandardCharsets.UTF_8
                )
            );

        StringBuilder hex =
            new StringBuilder();

        for (byte b : hash) {
            hex.append(
                String.format("%02x", b)
            );
        }
```

```


        String expectedSignature =
            hex.toString();

        return expectedSignature
            .equalsIgnoreCase(signature);
    }
}
```

Webhook Processing Flow

```
1. Receive the HTTP POST request
          ↓
2. Read the raw request body
          ↓
3. Read X-Webhook-Signature
          ↓
4. Calculate HMAC-SHA256
   using the webhook secret
          ↓
5. Compare calculated signature
   with the header
          ↓
6. Reject request if signature
   is invalid
          ↓
7. Parse JSON only after
   signature validation
          ↓
8. Process the webhook event
          ↓
```

```
9. Return HTTP 2xx response
```

Important:

Do not parse, format, reserialize, or otherwise modify the request body before calculating the signature. The signature is generated from the exact JSON payload sent by CittaEFS.

Keep your webhook secret private. Never expose it in frontend applications, client-side JavaScript, mobile apps, or public source code.

X-Webhook-Id uniquely identifies the webhook delivery and can be used to help prevent duplicate processing. X-Webhook-Event identifies the event type.

4. Submit E-Invoices

POST /api/integration/gen/invoices

Submit invoice line items directly.

Request Payload (IntegrationInvoiceLineDto)

```
[
  {
    // ========== REQUIRED FIELDS ==========
    "invoiceNumber": "INV2024001",
    "issueDate": "2026-07-28",

    // ========== CUSTOMER ==========
    // Provide EITHER customerCode OR customer.
    // Do NOT provide both.

    "customerCode": "CUST001",

    // OR use an inline customer:
    //
    // "customer": {
    //   "name": "ABC Limited",
    //   "tin": "123456789-0001",
    //   "email": "accounts@abclimited.com",
    //   "city": "Lagos",
    //   "country": "NG",
```

```
    //   "street": "12 Main Street",
    //   "postalCode": "100001"
    // },

    "itemName": "Laptop Pro",
    "itemDescription": "15-inch Laptop with 16GB RAM, 512GB SSD",
    "quantity": 2,
"unitPrice": 1500000.00,
"taxAmount": 217500.00,
"taxableAmount": 2900000.00,
    "hsOrServiceCode": "2620",
    "unitCode": "EA",
    "taxCategoryId": "STANDARD_VAT",
    "currencyCode": "NGN",
    "invoiceTypeCode": "381",

    // ========== OPTIONAL FIELDS ==========
    "lineNum": "1",
    "headerDiscount": 0.00,
    "headerCharges": 0.00,
```

```
    "lineDiscount": 100000.00,
     "useStateTax": false,
     "documentNumber": "PO-001",
     "billingReferenceIrns": [
       "IRN12345678901234567890"
     ],

     // ========== CUSTOM FIELDS ==========
     "customFields": {
       "Department": "IT",
       "CostCenter": "CC-001",
       "ProjectCode": "PRJ-2024-001"
     },

     // ========== METADATA ==========
     "metadata": {
       "SourceSystem": "ERP",
       "UserID": "user123",
       "BranchCode": "LAGOS-001",
       "SalesPerson": "John Doe"
```

```
     }
   }
]
```

Amount Calculation

```
TaxableAmount = (UnitPrice × Quantity) - LineDiscount

TaxAmount = TaxableAmount × 7.5%

Example:

UnitPrice     = 1,500,000.00
Quantity      = 2
LineDiscount  =   100,000.00

Gross Amount  = 1,500,000 × 2
              = 3,000,000.00

TaxableAmount = 3,000,000 - 100,000
              = 2,900,000.00

TaxAmount     = 2,900,000 × 7.5%
              = 217,500.00

Invoice Total = 2,900,000 + 217,500
```

```
              = 3,117,500.00
```

Inline Customer Example

```
[
  {
    "invoiceNumber": "INV2024002",
    "issueDate": "2026-07-28",

    "customer": {
      "name": "ABC Limited",
      "tin": "123456789-0001",
      "email": "accounts@abclimited.com",
      "city": "Lagos",
      "country": "NG",
      "street": "12 Main Street",
      "postalCode": "100001"
    },

    "itemName": "Laptop Pro",
    "itemDescription": "15-inch Laptop with 16GB RAM, 512GB SSD",
    "quantity": 2,
"unitPrice": 1500000.00,
"taxAmount": 217500.00,
```

```
"taxableAmount": 2900000.00,
    "hsOrServiceCode": "2620",

    "lineNum": "1",
    "unitCode": "EA",
    "taxCategoryId": "STANDARD_VAT",
    "currencyCode": "NGN",
    "invoiceTypeCode": "381",
    "headerDiscount": 0.00,
    "headerCharges": 0.00,
   "lineDiscount": 100000.00,
    "useStateTax": false,
    "documentNumber": "PO-001",
    "billingReferenceIrns": [
      "IRN12345678901234567890"
    ],

    "customFields": {
      "Department": "IT",
      "CostCenter": "CC-001",
```

```
      "ProjectCode": "PRJ-2024-001"
    },

    "metadata": {
      "SourceSystem": "ERP",
      "UserID": "user123",
      "BranchCode": "LAGOS-001",
      "SalesPerson": "John Doe"
    }
  }
]
```

Customer Options

Provide either customerCode OR customer, but not both.

customerCode
Existing customer code. Use this when the customer already exists in your CittaEFS customer records.

customer
Inline customer details. Use this when you do not want to reference an existing customer record.

Required inline customer fields: name, tin, email, city, country, street, postalCode.

Customer Rule
Provide either customerCode OR customer, but not both. If customer is provided, all customer fields are required and the invoice is automatically treated as B2B. The inline customer does not need to already exist as a customer record.

Response (Bulk Invoice Processing)

```
{
  "success": false,
  "totalInvoices": 3,
  "validCount": 2,
  "invalidCount": 1,

  "validInvoices": [
    "INV001",
    "INV003"
  ],

  "invalidInvoices": [
    "INV002"
  ],

  "validDocumentNumbers": [
    "PO001",
    "PO003"
  ],
```

```
  "metadata": {
    "SourceSystem": "ERP",
    "UserID": "user123",
    "BranchCode": "LAGOS-001",
    "SalesPerson": "John Doe"
  },

  "errors": [
    {
      "invoiceNumber": "INV002",
      "rowNumber": 2,
      "field": "TaxAmount",
      "error": "TaxAmount must equal TaxableAmount × applicable tax rate",
      "expectedValue": "217500.00",
      "actualValue": "225000.00"
    }
  ]
}
```

Response Fields

success: True when the entire batch completes without validation or processing failures.

totalInvoices: Total number of distinct invoices submitted.

validCount: Number of invoices that passed validation.

invalidCount: Number of invoices that failed validation.

validInvoices: Invoice numbers that passed validation.

invalidInvoices: Invoice numbers that failed validation.

validDocumentNumbers: Document numbers associated with successfully processed invoices.

metadata: Metadata associated with the submitted request.

errors: Detailed validation or processing errors.

Code Example

```
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;

using var client = new HttpClient();

client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", "YOUR_API_KEY");

var invoice = new[]
{
    new
    {
        invoiceNumber = "INV2024001",
        issueDate = "2026-07-28",

        customerCode = "CUST001",

        itemName = "Laptop Pro",
        itemDescription = "15-inch Laptop with 16GB RAM, 512GB SSD",
```

```
        quantity = 2,
        unitPrice = 1500000m,

        lineDiscount = 100000m,

        taxableAmount = 2900000m,
        taxAmount = 217500m,

        hsOrServiceCode = "2620",

        lineNum = "1",
        unitCode = "EA",
        taxCategoryId = "STANDARD_VAT",
        currencyCode = "NGN",
        invoiceTypeCode = "381",

        headerDiscount = 0m,
        headerCharges = 0m,
```

```
        useStateTax = false,

        documentNumber = "PO-001",

        billingReferenceIrns = new[]
        {
            "IRN12345678901234567890"
        },

        customFields = new Dictionary<string, object>
        {
            ["Department"] = "IT",
            ["CostCenter"] = "CC-001",
            ["ProjectCode"] = "PRJ-2024-001"
        },

        metadata = new Dictionary<string, object>
        {
            ["SourceSystem"] = "ERP",
            ["UserID"] = "user123",
```

```
            ["BranchCode"] = "LAGOS-001",
            ["SalesPerson"] = "John Doe"
        }
    }
};

var json = JsonSerializer.Serialize(invoice);

var content = new StringContent(
    json,
    Encoding.UTF8,
    "application/json"
);

var response = await client.PostAsync(
    "${ApiConfig.baseUrl}/api/integration/gen/invoices",
    content
);

var result = await response.Content.ReadAsStringAsync();
```

```
Console.WriteLine(result);
```

5. Resource Codes

Use these endpoints to retrieve the configured resource codes required when creating invoices.

```
Resource endpoints are public and do not require authentication.

Available resources:

GET /api/resources/countries
GET /api/resources/invoice-types
GET /api/resources/tax-categories
GET /api/resources/hs-codes
GET /api/resources/service-codes
```

Countries

GET /api/resources/countries
Retrieve the available country codes.

```
{
  "success": true,
  "message": "Countries retrieved successfully",
  "data": [
    {
      "alpha_2": "NG",
      "name": "Nigeria"
    }
  ]
}
```

Invoice Types

GET /api/resources/invoice-types

Retrieve the available invoice type codes.

```
{
  "success": true,
  "message": "Invoice types retrieved successfully",
  "data": [
    {
      "code": "381",
      "value": "Commercial Invoice"
    }
  ]
}
```

Tax Categories

GET /api/resources/tax-categories

Retrieve the available tax category codes.

```
{
  "success": true,
  "message": "Tax categories retrieved successfully",
  "data": [
    {
            "code": "STANDARD_VAT",
            "percent": "7.50%",
            "value": "Standard Value-Added Tax"
        },
  ]
}
```

HS Codes

GET /api/resources/hs-codes

Retrieve the available HS codes.

```
{
  "success": true,
  "message": "HS codes retrieved successfully",
  "data": [
        {
      "hscode": "0101.21",
      "description": "Horses"
    }
  ]
}
```

Service Codes

GET /api/resources/service-codes
Retrieve the available service codes.

```
{
  "success": true,
  "message": "Service codes retrieved successfully",
  "data": [
    {
            "code": "1104",
            "description": "Manufacture of soft drinks; production of mineral waters and other bottled
waters"
        },
  ]
}
```

Usage in Invoice Submission

The returned resource codes can be used when submitting an invoice. invoiceTypeCode must contain a valid configured invoice type, taxCategoryId must contain a valid configured tax category, and hsOrServiceCode must contain a valid HS or Service Code.

```
Example:

{
  "invoiceTypeCode": "381",
  "taxCategoryId": "STANDARD_VAT",
  "hsOrServiceCode": "2620",
  "currencyCode": "NGN"
}
```

6. Validation & Processing Rules

Invoices are validated individually within the submitted batch. Invoices that pass validation are included in validInvoices and continue to processing. Invoices that fail validation are included in invalidInvoices and do not continue to processing. Processing errors are reported separately through the errors collection. One invalid invoice does not prevent other invoices in the same batch from being processed.

```
Batch Processing

A batch can contain both valid and invalid invoices.

Each invoice is validated independently.

VALID:
    Included in validInvoices
    Continues to processing

INVALID:
    Included in invalidInvoices
    Does not continue to processing

PROCESSING FAILURE:
    Reported through errors

One invalid invoice does not prevent other
valid invoices in the same batch from being processed.
```

Duplicate Invoice Numbers

Invoice numbers must be unique for the company. An invoice number that has already been posted to the archive (WM_001_EI_AR) is considered a duplicate and will fail validation.

```
Duplicate validation:

Existing invoice:
    INV2024001

New request:
    INV2024001

Result:
    INVALID

Error:
    Invoice Number already posted

Expected:
    Unique invoice number
```

Invoice numbers must contain only letters A-Z and numbers 0-9 and must be unique for the company.

Invoice Validation Rules

Field	Validation
Invoice Number	Required, unique, A-Z and 0-9 only
Issue Date	Required, YYYY-MM-DD format
Customer	customerCode OR customer object required, but not both
Customer Object	Requires Name, Tin, Email, City, Country, Street and PostalCode
Item Name	Required

Item Description	Required
Quantity	Must be greater than 0
Unit Price	Must be greater than 0
Taxable Amount	Must be greater than 0 and equal to (UnitPrice × Quantity) - LineDiscount
Tax Amount	Cannot be negative and must match the applicable tax rate
Unit Code	Required
Invoice Type	Required and must be a valid configured invoice type
HS / Service Code	Must be a valid configured HS or Service Code
Currency	Must be a valid configured currency code
Tax Category	Must be a valid configured tax category
Billing Reference	Required for invoice types 380, 384 and 393
Billing Reference IRN	Must exist in the company archive
Self-Billed TIN	Required for invoice types 385 and 393

Customer Validation

```
A customer must be supplied using ONE of:

1. customerCode

OR

2. customer object

Do not provide both.

Valid:

{
  "customerCode": "CUST001"
}

OR:

{
  "customer": {
```

```
    "name": "ABC Limited",
    "tin": "123456789-0001",
    "email": "accounts@abclimited.com",
    "city": "Lagos",
    "country": "NG",
    "street": "12 Main Street",
    "postalCode": "100001"
  }
}

Invalid:

{
  "customerCode": "CUST001",
  "customer": {
    ...
  }
}
```

When an inline customer object is used, all customer fields are required and the invoice is treated as B2B.

Amount Validation

```
TaxableAmount must equal:

(UnitPrice × Quantity) - LineDiscount

TaxAmount must equal:

TaxableAmount × applicable tax rate

Example:

UnitPrice     = 1,500,000
Quantity      = 2
LineDiscount  =   100,000

TaxableAmount = (1,500,000 × 2) - 100,000
              = 2,900,000

VAT           = 2,900,000 × 7.5%
              = 217,500
```

```
Invoice Total = 2,900,000 + 217,500
              = 3,117,500
```

Billing Reference Validation

Invoice types 380, 384 and 393 require at least one billing reference IRN. Each supplied IRN must exist in the company archive.

```
InvoiceTypeCode: 380

BillingReferenceIrns:
[
  "IRN12345678901234567890"
]

The IRN must exist in the archived invoices
for the authenticated company.
```

Self-Billed Invoice Validation

Self-billed invoice types 385 and 393 require a valid customer TIN and customer name/code.

```
Self-billed invoice types:

385
393

Required:

Customer TIN
    - Must not be empty
    - Must not be "00000-000"

Customer identity
    - CustomerCode
      OR
    - Customer.Name
```

Processing Statuses

```
Standard Processing:

DRAFT
    Invoice created as a draft when manual processing is enabled.

QUEUED
    Invoice queued for automatic processing.
```

Existing Invoice Status Handling

When processing a previously submitted invoice number, existing DRAFT and SIGN_FAILED records can be removed and recreated. Invoices with SIGNED, TRANSMITTED or TRANSMIT_FAILED status cannot be overwritten during this processing step.

```
Deletable existing statuses:

DRAFT
SIGN_FAILED

Non-deletable statuses:

SIGNED
TRANSMITTED
TRANSMIT_FAILED

If a non-deletable invoice exists:

success: false

status: "FAILED"

message:
"Invoice already exists with non-deletable status
(SIGNED/TRANSMITTED/TRANSMIT_FAILED)"
```

Validation Errors

Validation errors are stored and, when webhooks are enabled, a VALIDATION_FAILED webhook is also sent for the affected invoice.

```
Validation error information includes:

invoiceNumber
request
documentNumber
rowNumber
field
error
expectedValue
actualValue
```

Mixed Valid & Invalid Batch Example

A single request can contain multiple invoices. Each invoice is validated independently.

```
Request:

[
  {
    "invoiceNumber": "INV001",
    ...
  },
  {
    "invoiceNumber": "INV002",
    ...
  },
  {
    "invoiceNumber": "INV003",
    ...
  }
]

Validation:

INV001 ☒ VALID
```

```
INV002 ☒ INVALID
INV003 ☒ VALID

Processing:

INV001 ☒ processed
INV002 ☒ rejected
INV003 ☒ processed
```

```
Response:

{
  "success": false,
  "totalInvoices": 3,
  "validCount": 2,
  "invalidCount": 1,

  "validInvoices": [
    "INV001",
    "INV003"
  ],

  "invalidInvoices": [
    "INV002"
  ],

  "validDocumentNumbers": [
    "PO001",
    "PO003"
```

```
  ],

  "metadata": {
    "SourceSystem": "ERP",
    "UserID": "user123",
    "BranchCode": "LAGOS-001",
    "SalesPerson": "John Doe"
  },

  "errors": [
    {
      "invoiceNumber": "INV002",
      "rowNumber": 2,
      "field": "TaxAmount",
      "error": "TaxAmount must equal TaxableAmount × applicable tax rate",
      "expectedValue": "217500.00",
      "actualValue": "225000.00"
    }
  ]
}
```

Important

The response distinguishes validation results from processing results. validInvoices contains invoices that passed validation, while invalidInvoices contains invoices that failed validation. Processing failures are reported through errors, and successfully processed document numbers are returned in validDocumentNumbers. The response also includes metadata associated with the submitted request. If one or more invoices fail validation or processing, success is false and the response contains the relevant error details.

7. Get Archive Records

GET /api/einvoice/archive

Fetch successfully transmitted invoices.

```
var response = await client.GetAsync(
    "${ApiConfig.baseUrl}/api/einvoice/archive?fromDate=2026-07-01&toDate=2026-07-28&paymentStatus=PAID");
```

8. Error Monitoring

Available endpoints:

GET /api/einvoice/errors/validation
GET /api/einvoice/errors/transmit
GET /api/einvoice/errors/sign

```
var response = await client.GetAsync(
    "${ApiConfig.baseUrl}/api/einvoice/errors/validation?fromDate=2026-07-01&toDate=2026-07-28");
```

9. Payment Management

PATCH /api/einvoice/update/{irn}

PATCH /api/einvoice/bulk/update

```
{
  "payment_status": "PAID",
  "reference": "BANKREF123456"
}
```

```
var payload = new BulkUpdatePaymentStatusDto
{
    Items = new List<BulkPaymentStatusItem>
    {
        new BulkPaymentStatusItem
        {
            Irn = "IRN1234567890",
            PaymentStatus = "PAID",
            Reference = "BANKREF123"
        }
    }
};

var json = JsonSerializer.Serialize(payload);
var content = new StringContent(json, Encoding.UTF8, "application/json");

var response = await client.PatchAsync(
    "${ApiConfig.baseUrl}/api/einvoice/bulk/update",
    content);
```